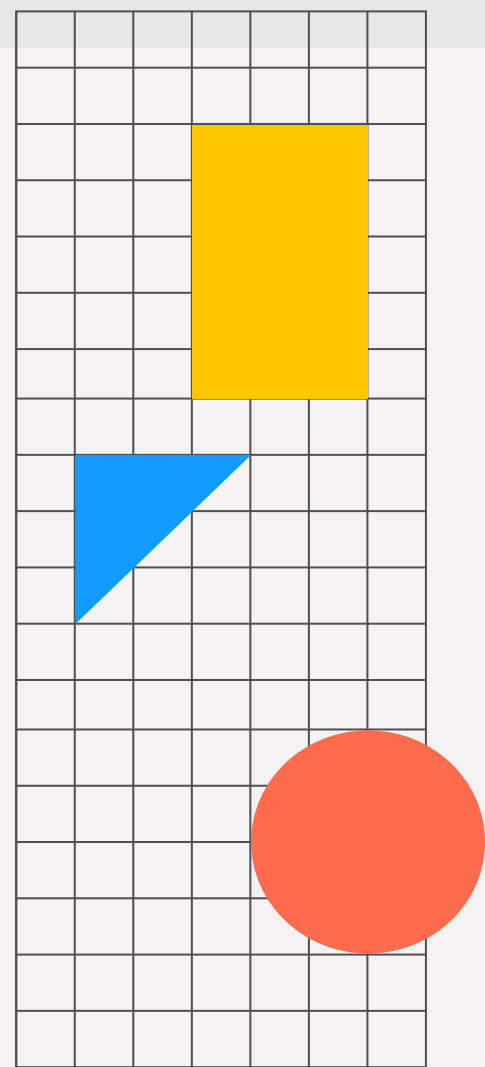


Practical Design and Development of AI Applications



Prompting Techniques

Prompting is the process of structuring inputs (instructions, context, and examples) to guide a language model toward producing desired outputs.

A prompt can include:

- Instructions (what to do)
- Context (background information)
- Input data
- Output format constraints

Effective prompting is critical because:

- LLMs are instruction-following systems, not deterministic programs
- Output quality depends heavily on how the problem is framed

Prompting Techniques : Zero-Shot Prompting

How it Works

- The model relies entirely on its pre-trained knowledge
- Instructions must be clear and unambiguous

Example

Classify the sentiment of this sentence:

"I love this product!"

Potential Answer: Positive /
Negative / Neutral

Advantages

- Simple and fast
- No need to prepare examples
- Works well for common tasks

Challenges

- Performance drops for complex tasks
- Ambiguity in instructions can lead to inconsistent outputs

Zero-shot prompting involves giving the model a task without any examples.

Prompting Techniques : Few-Shot Prompting

How it Works

- The model learns the pattern from examples
- Applies the same pattern to new inputs

Example

Sentence: "I love this!"
Sentiment: Positive

Sentence: "This is terrible."
Sentiment: Negative

Sentence: "It's okay."
Sentiment: ?

Advantages

- Improves accuracy significantly
- Helps enforce formatting and style

Challenges

- Token limit constraints
- Example selection is critical
- Can introduce bias

Few-shot prompting provides a small number of examples to guide the model.

Prompting Techniques : Chain-of-Thought Prompting

How it Works

- Prompts include intermediate reasoning steps. Model generates a reasoning chain
- Break a complex problem across multiple prompts instead of one, use each prompt's output as the next prompt's input, forming a chain that progressively solves the task.

Example

Question: If a train travels 60 km in 1 hour, how far in 3 hours?

Think step by step.

Advantages

- Improves reasoning accuracy
- Useful for math, logic, and multi-step problems
- Breaks down problems
- Better planning for actions

Challenges

- Increases token usage
- May produce incorrect reasoning if not guided properly

Encourages the model to reason step-by-step before giving the final answer.

Prompting Techniques : **Multimodal Prompting**

How it Works

- Combine visual/textual context in prompt

Example

Image + question: "What is happening here?"

Advantages

- Richer understanding
- Enables real-world applications

Challenges

- More complex models
- Interpretation errors

Uses multiple input types (text, image, audio).

Prompting Techniques : Summary

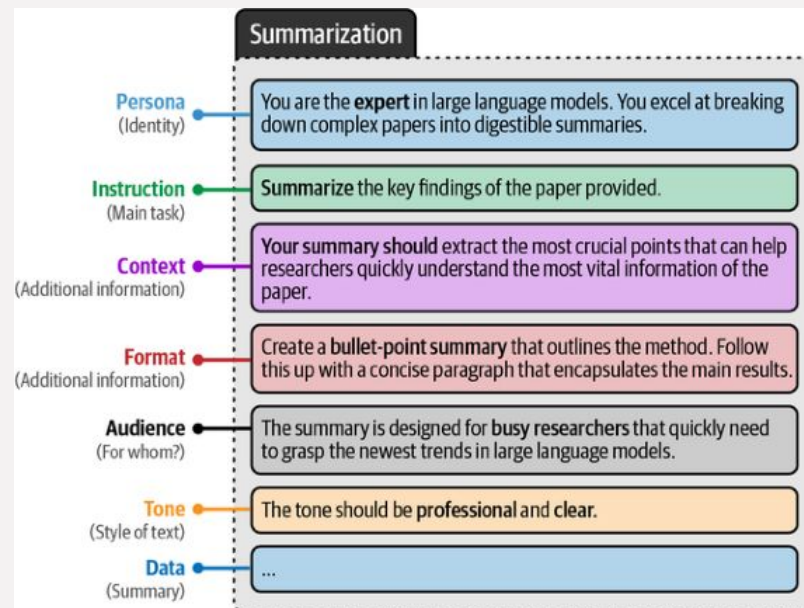
Persona : Describe what role the LLM should take on.

Instruction : The task itself. Make sure this is as specific as possible. We do not want to leave much room for interpretation.

Context : Additional information describing the context of the problem or task.

Format : Without it, the LLM will come up with a format itself, which is troublesome in automated systems.

Tone : The tone of voice the LLM should use in the generated text. If you are writing a formal email to your boss, you might not want to use an informal tone of voice.



Uses multiple input types (text, image, audio).

Context Windows

What is it ?

- A context window is the maximum number of tokens an LLM can process in a single request. It includes the prompt, instructions, conversation history, and output essentially the model's working memory.

How it works

If a model has a 100K token context window, one can input ~80K tokens and get ~20K tokens of output Or use smaller input and larger output, as long as total $\leq 100K$

Why it matters ?

It determines how much information the model can use to:

- Understand context
- Maintain coherence
- Reason across inputs (documents, conversations, code)

Challenges

Limited Capacity : Long inputs may not fit → requires chunking or summarization.

Input vs Output Trade-off : More input reduces space for output, limiting response depth.

Attention Dilution : Important information can get lost in large contexts ("lost in the middle").

Cost & Latency : Larger context increases compute cost and slows response time.

RAG: Retrieval-Augmented Generation



The Core Idea

RAG is a technique where models retrieve relevant data from **external knowledge bases** to generate answers.

- **Beyond Memory:** Doesn't just rely on static training weights.
- **Dynamic Access:** Connects LLMs to private or real-time data.
- **Fact-Grounded:** Uses citations from ground-truth sources.

LLM + Knowledge Retrieval = Superior, Fact-Grounded Answers

RAG: Bridging the LLM Knowledge Gap

RAG augments LLMs with **external data sources**, providing a lightweight alternative to intensive pre-training for proprietary data access.

Private Data

Secure Access

Connects LLMs to proprietary datasets not seen during training.

Accuracy

No Hallucinations

Forces models to cite ground-truth retrieval sources.

Freshness

Real-time Knowledge

Answers questions about events occurring after pre-training.

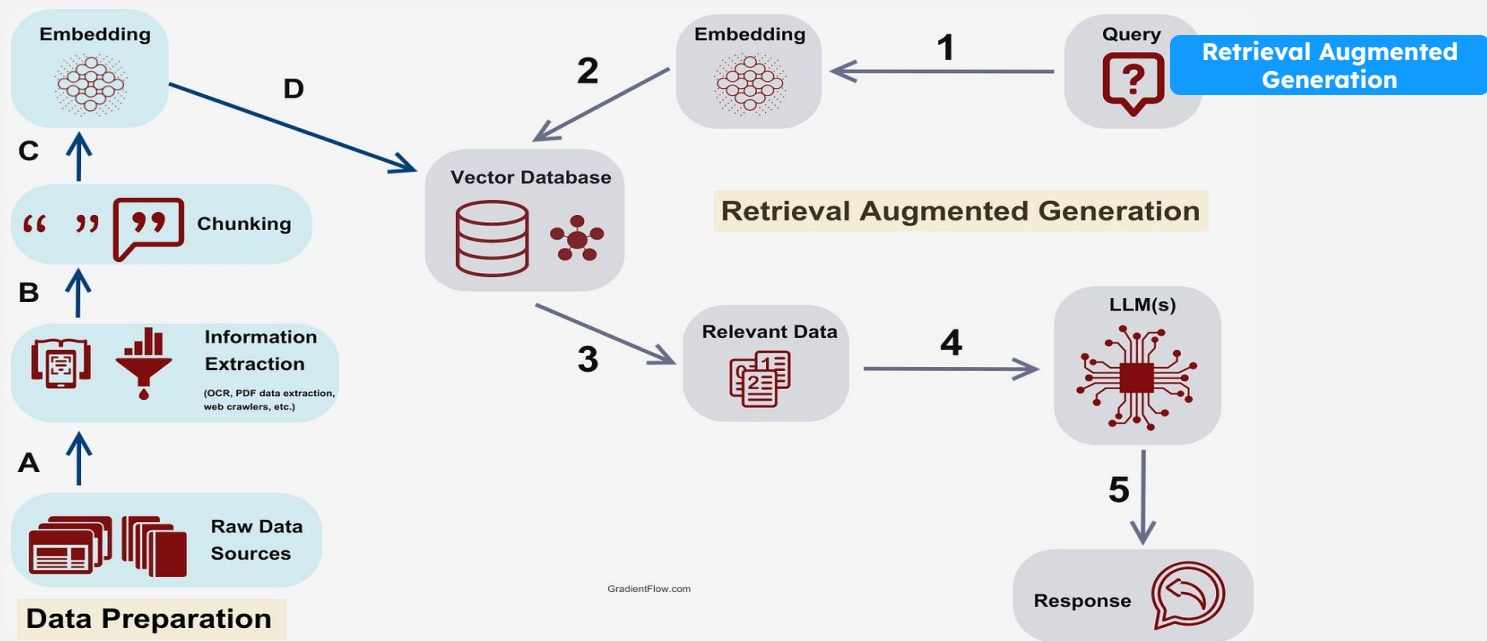
Long-Tail Queries

Niche Expertise

Handles rare entities often missing from generic training sets.

RAG transforms LLMs from static calculators into dynamic, fact-grounded knowledge systems.

RAG: General Architecture & Data Flow



A 5-step process connecting raw data sources to fact-grounded LLM responses

RAG: The Core Competitive Advantages

Accuracy

Reduced Hallucinations

Provides answers grounded in real-world documents.

Agility

Instant Updates

Refreshes knowledge without retraining models.

Security

Privacy First

Seamlessly integrates with private, proprietary data.

Efficiency

Cost Savings

Eliminates need for expensive fine-tuning.

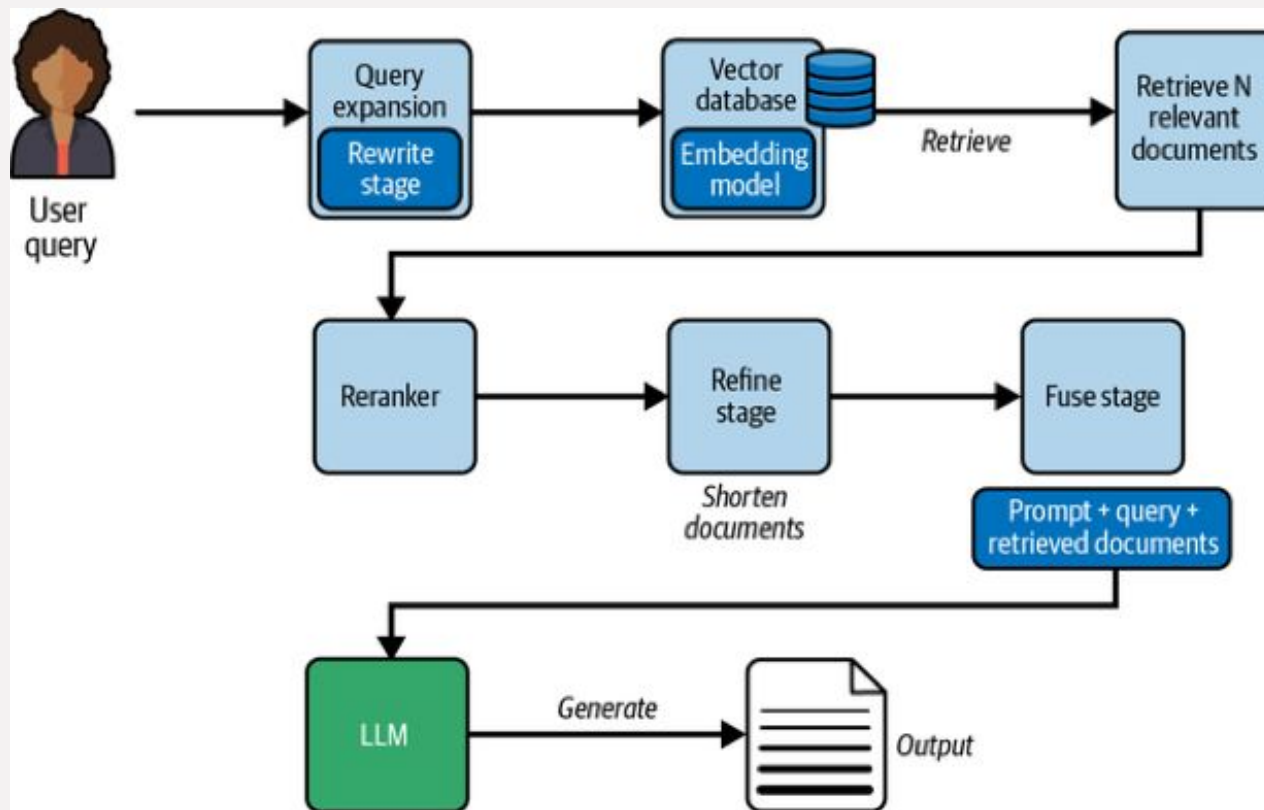
Trust

Transparency

Cites sources for verifiable and clear answers.

RAG empowers LLMs with verifiable, low-cost, and real-time domain expertise.

RAG: Pipeline Stages



Stages

1. User query
2. Query Expansion
3. Vector Database
4. Data retrieval
5. Reranking
5. Refine stage
6. Fuse stage
7. LLM Interaction
8. Output

RAG Stages: User Query

What Happens ?

- User provides input in natural language
 - Example: *"What are the side effects of drug X?"*
- Query may be:
 - Informational
 - Analytical
 - Conversational
 - Multi-part or ambiguous

Challenges

- Ambiguity
- Underspecified queries
- Domain-specific jargon

Processing Steps

- Input normalization:
 - Lowercasing
 - Removing noise (optional)
- Language detection (for multilingual systems)
- Context attachment

This is the **entry point** of the RAG pipeline.

RAG Stages: Query Expansion

Why it is needed ?

- User queries are often **short, vague, or incomplete**
- Retrieval systems work better with **rich semantic signals**

Synonym Expansion

Add equivalent terms

Example:

"car" → "automobile", "vehicle"

Semantic Expansion

LLM rewrites query into multiple variations

Example: "effects of caffeine"

- "impact of caffeine"
- "side effects of caffeine consumption"

HyDE - Hypothetical Doc Embedding

Generate a "hypothetical" document from another LLM that represents an ideal answer to a query

Embed response instead of query

Rephrases & Enhances the original query to improve retrieval quality.

RAG Stages: **Embed & Vector Database**

What is stored ?

- Document chunks (not full documents)
- Metadata:
 - Source
 - Timestamp
 - Author
- Embeddings (dense vectors)

Key Capabilities ?

- Fast similarity search (cosine, dot product)
- Filtering via metadata
- Scalability for large corpora

Stores embeddings of documents for efficient **similarity search**.

RAG Stages: Data Retrieval

Importance

- Retrieval determines **what knowledge the LLM sees**, making it one of the highest-impact stages in the pipeline.
- Fetches relevant documents based on query embeddings.

General Steps

- Convert query → embedding vector
- Perform similarity search in vector DB
- Retrieve top-K results

Dense Retrieval

Sparse Retrieval

Hybrid Retrieval

RAG Stages: **Sparse Data Retrieval**

Core Idea

- Uses **term frequency and exact keyword matching**
- Non semantic, based on fetching common words or keywords

Common Algorithms

- BM25 (most widely used)
 - Best Matching 25
 - Default algo in ElasticSearch & SOLR
- Ranking function based on:
 - Term frequency (TF)
 - Inverse document frequency (IDF)
 - Document length normalization

Strengths

- Excellent for Exact matches & Rare keywords (e.g., error codes, IDs)
- Deterministic and interpretable
- Logs, code search, legal text
- Queries with precise terminology

Weaknesses

- Fails with:
 - Synonyms
 - Semantic meaning
- Poor performance for:
 - Natural language queries

RAG Stages: Dense Data Retrieval

Core Idea

- Uses vector embeddings to capture semantic meaning
- Retrieves documents based on conceptual similarity, not exact words

Steps

- Query $\rightarrow$ embedding vector q
- Documents $\rightarrow$ embedding vectors $d_1, d_2, \dots, d_n$
- Compute similarity:
 - Cosine similarity (most common)
 - Dot product
- Retrieve top-K documents with highest similarity

Strengths

- Captures meaning, not just keywords
- Handles:
 - Synonyms ("car" vs "automobile")
 - Paraphrasing
 - Natural language queries
- Works well for:
 - Conversational AI
 - Open-ended questions

Weaknesses

- May miss exact keyword matches
- Sensitive to:
 - Embedding model quality
- Can retrieve:
 - Semantically similar but factually irrelevant docs

RAG Stages: Hybrid Data Retrieval

Core Idea

- Combine **dense + sparse retrieval**, get the best of both worlds
- Higher recall + higher precision

Fusion Techniques

Score-level fusion

- $\text{final_score} = \alpha * \text{dense_score} + \beta * \text{sparse_score}$

Reciprocal Rank Fusion (RRF)

Combines rankings instead of scores

- $\text{RRF} = \sum (1 / (k + \text{rank}_i))$
- Very robust in practice

Strengths

Handles:

- Synonyms AND exact keywords

Improves:

- Recall@K significantly
- Widely used in production systems

Weaknesses

- Slightly higher latency
- More system complexity

RAG Stages: Reranking

Why it is needed ?

- Initial retrieval prioritizes speed over precision
- Reranking optimizes for relevance

Step 1: Candidate selection

Take top-K from the available ranked matches

(e.g., 20-100 documents)

Step 2: Scoring with cross-encoder

Use a cross-attention model like BERT

- Input: (query + document)
- Output: Relevance score

Step 3: Sorting

Rank documents by relevance score

Improves quality of retrieved results. Ranks & Retrieves Top-N highly relevant documents

RAG Stages: **Refine**

Purpose

- Clean, filter, and optimize context for LLM

Deduplication

- Remove repeated chunks

Filtering

- Remove irrelevant or low-confidence chunks

Compression

- Summarize long chunks
- Extract key sentences

Context window optimization

- Fit content within LLM token limits

Rewriting chunks

- Normalize language
- Remove noise

Improves the quality of retrieved content before generation.

RAG Stages: Fuse

Components fused

- Original user query
- Expanded query (optional)
- Retrieved documents
- System instructions
- Chat history (if conversational)

Challenges

- Token limits
- Context dilution
- Prompt injection risks

Prompt Structure

System: You are a helpful assistant...

Context:

[Doc 1]

[Doc 2]

...

User Query:

"What are the side effects of X?"

Instructions:

- Answer only from context
- Cite sources if possible

Combines all components into a final prompt for the LLM.

RAG Stages: Passing Data to LLM & Output

What happens ?

- LLM processes:
 - Context + Query
 - Instructions
- Applies:
 - Attention mechanism
 - Probabilistic token generation
- LLM generates the final response.
 - Token-by-token generation
 - Stops at:
 - End-of-sequence
 - Token limit

Post-processing

a) Formatting

- Markdown
- Bullet points

b) Validation

- Fact-checking
- Rule enforcement

c) Guardrails

- Toxicity filtering & Policy compliance

Final prompt is sent to the language model.

RAG: Understanding Key Disadvantages

Retrieval Errors

"Garbage In, Garbage Out"

Retrieving incorrect or irrelevant documents leads directly to hallucinated or inaccurate LLM responses.

Context Limits

Token Bottlenecks

LLMs have finite window sizes; providing too much context can exceed token limits or dilute focus.

Chunking Strategy

Information Loss

Sub-optimal text splitting (bad chunking) results in fragmented context and loss of semantic meaning.

System Latency

Execution Overhead

Multi-step pipelines (retrieval, ranking, generation) inevitably increase end-to-end response time.

Successful RAG implementation requires balancing retrieval quality, context management, and speed.

Advanced RAG Techniques : Multi Step Self RAG

Multi-Step Self-RAG is an advanced RAG approach that combines:

- **Iterative retrieval (multi-step reasoning) & Self-evaluation (reflection and correction)**

It enables the system to **retrieve, generate, critique, and refine answers in multiple cycles** instead of a single pass.

Steps Involved

Step 1: User query is processed and optionally expanded

Step 2: Initial retrieval fetches relevant documents

Step 3: LLM generates a **partial answer or reasoning**

Step 4: A **self-evaluation step (critic)** checks:

- Completeness, Relevance, Missing information

Step 5: System identifies gaps and **refines the query**

Step 6: Performs **additional retrieval** based on refined queries

Step 7: Merges new context and **regenerates the answer**

Step 8: Loop continues until Answer quality is sufficient **or** Iteration limit is reached

Advanced RAG Techniques : Multi Step Self RAG

Architecture Components

1. Retriever

Fetches documents at each iteration

2. Generator (LLM)

Produces answers and reasoning

3. Critic / Evaluator

Evaluates:

- Accuracy
- Completeness & Relevance

4. Controller (Orchestrator)

Decides: When to stop & When to retrieve again

Types of Self-Evaluation

a) Answer Quality Check

Is the answer coherent and relevant?

b) Evidence Grounding Check

Does the answer rely on retrieved context?

c) Completeness Check

Are all aspects of the query addressed?

d) Contradiction Detection

Conflicts between sources or reasoning

Advanced RAG Techniques : Multi Step Self RAG

Benefits

Improved Answer Quality

Iterative refinement leads to more accurate and complete responses

Reduced Hallucination

Self-check ensures grounding in retrieved data

Adaptive Intelligence

System dynamically decides
What to retrieve & When to stop

Challenges

Increased Latency

Multiple retrieval + generation cycles

Higher Cost

Due to More LLM calls

Error Propagation

Incorrect intermediate steps can mislead future iterations

Control Complexity

Requires orchestration logic

Optimization Techniques

Early Stopping

Stop when confidence threshold is reached

Iteration Limits

Prevent infinite loops

Caching

Cache Retrieval results & Intermediate answers

Selective Retrieval

Skip retrieval if context is sufficient

Retrieve → Generate → Critique → Refine → Retrieve again → Repeat

Explainable Retrieval in Document Search

Traditional document retrieval systems often work as black boxes, providing results without explaining why they were chosen. This lack of transparency can be problematic in scenarios where understanding the reasoning behind the results is crucial.

Benefits of a Explainable Retrieval Setup

Transparency: Users can understand why specific documents were retrieved.

Trust: Explanations build user confidence in the system's results.

Learning: Users can gain insights into the relationships between queries and documents.

Debugging: Easier to identify and correct issues in the retrieval process.

Customization: The explanation prompt can be tailored for different use cases or domains.

The Explainable Retriever represents a significant step towards more interpretable and trustworthy information retrieval systems. By providing natural language explanations alongside retrieved documents, it bridges the gap between powerful vector-based search techniques and human understanding. Reasoning behind information retrieval is as important as the retrieved information itself, such as legal research, medical information systems, and educational tools.

Advanced RAG Techniques : Graph RAG

- Graph RAG is an advanced RAG approach that combines vector-based retrieval with knowledge graphs to leverage relationships between entities for better reasoning.
- A **knowledge graph** is a structured, semantic network that connects real-world entities—people, places, concepts, or objects—and their relationships. Using a graph database, it stores information as nodes (entities) and edges (relationships), transforming raw data into a machine-understandable web of context.
- Traditional RAG finds relevant information, Graph RAG connects the dots between them.

How It Works

- Extract entities from the query and documents
- Retrieve relevant information using vector search
- Traverse the knowledge graph to find connected entities and relationships
- Combine retrieved text + graph insights into context

Key Difference to Traditional RAG

Retrieves connected information via entity relationships

- **Focuses on:** “How are these pieces of information related?”
- **Uses:** Knowledge graphs + vector search
- **Strength:** Enables multi-hop reasoning across entities

Advanced RAG Techniques : Graph RAG Method Details

Document Processing

- Input documents are split into manageable chunks. Each chunk is embedded using a language model.
- A vector store is created from these embeddings for efficient similarity search.

Knowledge Graph Construction

- Graph nodes are created for each text chunk.
- Edges are added between nodes based on semantic similarity and shared concepts.
- Edge weights are calculated to represent the strength of relationships.

Query Processing

- A priority queue is initialized with the nodes corresponding to the most relevant documents.
- The system employs a Dijkstra-like algorithm to traverse the knowledge graph
- This process continues until a complete answer is found / priority queue is exhausted.

Visualization

- The knowledge graph is visualized with nodes representing text chunks and edges representing relationships.
- Edge colors indicate the strength of relationships
- The traversal path taken to answer a query is highlighted with curved, dashed arrows.

Advanced RAG Techniques : Graph RAG Benefits

Improved Context Awareness: By representing knowledge as a graph, the system can maintain better context and make connections across different parts of the input documents.

Enhanced Retrieval: The graph structure allows for more intelligent retrieval of information, going beyond simple keyword matching.

Explainable Results: The visualization of the graph and traversal path provides insight into how the system arrived at its answer, improving transparency and trust.

Flexible Knowledge Representation: The graph structure can easily incorporate new information and relationships as they become available.

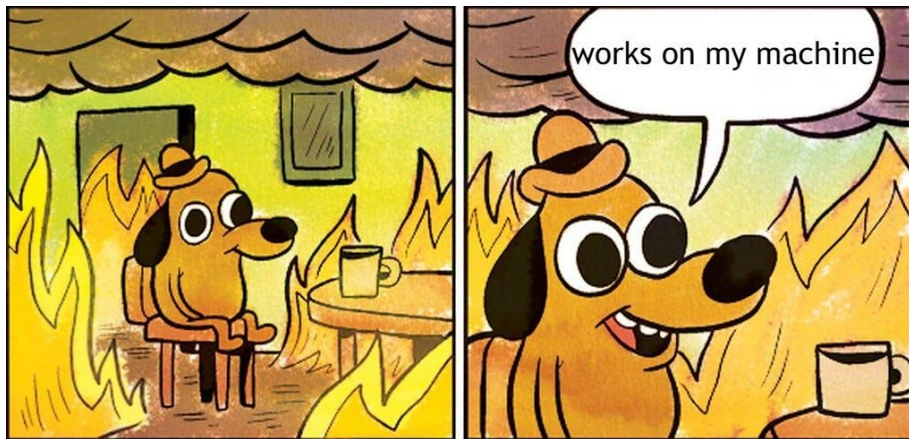
Efficient Information Traversal: The weighted edges in the graph allow the system to prioritize the most relevant information pathways when answering queries.

By incorporating a graph-based knowledge representation and intelligent traversal mechanisms, it offers improved context awareness, more accurate retrieval, and enhanced explainability.

Docker

Docker lets you bundle your application along with everything it needs (code, libraries, dependencies, runtime) into a single portable unit called a container.

Think of it like: “Run this app anywhere, and it behaves exactly the same.”



Docker : What is a container ?

A container is a lightweight, isolated environment where your application runs.

- Includes: app code + runtime + system tools + libraries
- Runs consistently across environments (dev, test, prod)
- Much faster and smaller than virtual machines

Why Docker matters

- Dependency conflicts across environments
- Difficult deployments

With Docker

- Consistent environments
- Easy deployment
- Faster onboarding
- Scalable systems

Docker : Containers VS Virtual Machines ?

Architecture

- **Docker:** Shares the host OS kernel (containerized apps run on the same OS)
- **VMs:** Each VM has its own full OS on top of a hypervisor

Resource Usage

- **Docker:** Lightweight (MBs), uses fewer CPU/RAM
- **VMs:** Heavy (GBs), requires more system resources

Startup Time

- **Docker:** Starts in seconds
- **VMs:** Takes minutes (boots full OS)

Isolation

- **Docker:** Process-level isolation (less strict, but efficient)
- **VMs:** Full isolation (more secure, separate OS per VM)

Docker : Key Concepts

- **Image** : immutable, layered filesystem snapshot (read-only); like a class
- **Dockerfile** : declarative recipe for building an image, one instruction per layer
- **Container** : a running (or stopped) instance of an image; like an object
 - Containers add a writable layer on top; changes vanish when container is removed
- **Volume** : persistent storage mounted into the container; survives container removal
- **Network** : virtual network; by default, containers are isolated from host & each other
- **Layer** : each instruction produces a cached, content-addressed overlay layer
- **Docker Hub** : Public registry to store and share images

Docker : Essential Commands

docker pull python:3.12-slim # download image from registry

docker run -it python:3.12-slim bash # interactive shell in a container

docker run -d -p 8080:80 nginx # detached, map host:container port

docker ps / docker ps -a # running / all containers

docker stop <id> && docker rm <id> # stop then remove

docker logs -f <id> # stream container stdout/stderr

docker exec -it <id> bash # shell into running container

docker inspect <id> # full JSON metadata (IP, mounts, etc.)

docker stats # live CPU/mem/net/disk per container

docker system prune -a # remove all unused images + containers

docker image history myapp # show layer sizes + build commands

Docker : Writing & Running Basic Dockerfile

```
FROM python:3.12-slim      # base image — always explicit tag, never 'latest'
```

```
WORKDIR /app              # sets CWD for all subsequent instructions
```

```
COPY requirements.txt .    # copy dep file BEFORE source (cache efficiency)
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .                  # copy source code (invalidates cache here only)
```

```
ENV PORT=8080             # build-time + runtime env var
```

```
EXPOSE 8080               # documentation only — does not publish port
```

```
USER nobody               # drop root; run as unprivileged user
```

```
CMD ["python", "-m", "uvicorn", "app:app", "--host", "0.0.0.0"]
```

Build & run

```
docker build -t myapp:1.0 .    # -t = name:tag
```

```
docker run -p 8080:8080 myapp:1.0
```

Docker : Writing & Running Basic Dockerfile

Dockerfile

A Dockerfile defines how to create a Docker image.

- Contains step-by-step instructions.
- Used during docker build
- Produces a reusable image

Example responsibilities:

Set base image (FROM)
Install dependencies (RUN)
Copy code (COPY)
Define default command (CMD)

Think of it as: “How do I package my app?”

docker-compose.yml

A docker-compose.yml defines how to run one or more containers together.

- Manages multiple services
- Handles networking, volumes, environment variables

Example responsibilities:

- Define services
- Map ports
- Set environment variables
- Define dependencies

Think of it as: “How do I run my entire system?”

docker-entrypoint.sh

A docker-entrypoint.sh is a shell script that runs when a container starts.

- Executes inside the container
- Used for initialization logic before the app starts

Example responsibilities:

- Wait for DB to be ready
- Run migrations
- Set environment configs
- Start the main process

Think of it as: “What should happen when the container starts?”

Git & GitHub : What is Git ?

Git is a **distributed version control system (DVCS)** used to track changes in code and collaborate efficiently.

Key Concepts

- **Repository (repo):** Project folder tracked by Git
- **Commit:** Snapshot of changes
- **Branch:** Independent line of development
- **Merge:** Combining branches
- **Clone:** Copy of a remote repository
- **Pull / Push:** Sync local ↔ remote

Why Git Matters

- Tracks history of code changes
- Enables collaboration without overwriting work
- Supports branching for experimentation

Git & GitHub : What is GitHub ?

- GitHub is a cloud-based platform built on Git that adds collaboration, UI, and automation features.
- Hosts 420 M+ repositories; owned by Microsoft since 2018

Core Idea

- Git = tool
- GitHub = platform using that tool
- Git is a DAG (Directed Acyclic Graph) of content-addressed snapshot objects

Key GitHub Features

- **Repository Hosting** : Store and manage code remotely
- **Pull Requests (PRs)** : Propose and review changes before merging
- **Issues** : Track bugs, features, tasks
- **Actions (CI/CD)** : Automate workflows (build, test, deploy)
- **Forking** : Copy someone else's repo to experiment
- **Code Review Tools** : Inline comments, approvals, change tracking

Git & GitHub : Essential Commands

Step 1: Initialize Git

```
</> Bash  
git init
```

Step 2: Add files

```
</> Bash  
git add .
```

Step 3: Commit

```
</> Bash  
git commit -m "Initial commit"
```

Step 4: Create GitHub Repo

- Go to GitHub → New Repository

Step 5: Connect Local to Remote

```
</> Bash  
git remote add origin <repo-url>
```

Step 6: Push Code

```
</> Bash  
git push -u origin main
```

Git & GitHub : Branches & Merging

Create Branch

 Bash

```
git checkout -b feature-branch
```

Switch Branch

 Bash

```
git checkout main
```

1. Fast-Forward Merge

- Linear history
- No divergence

 Bash

```
git merge feature-branch
```



2. Three-Way Merge

- Creates a merge commit
- Used when branches diverge

3. Squash Merge

- Combines multiple commits into one

 Bash

```
git merge --squash feature-branch
```



4. Rebase

- Rewrites history for cleaner logs

 Bash

```
git rebase main
```



Git & GitHub : Git Hooks

Git Hooks are scripts that automatically run when specific Git events occur (like committing, pushing, merging, etc.).

- They help enforce rules, automate checks, and improve code quality before or after actions happen.

Pre-Commit (before commit)

Prevent bad code from being committed, Stops commit if tests fail

Commit-Msg (validate message)

Enforce commit format, Ensures proper commit messages

Pre-Push (before push)

Prevent broken code from being pushed, Stops push if build fails

Post-Merge (after merge)

Run setup after pulling changes

Git & GitHub : **GitHub Actions & Secrets**

GitHub Actions is a CI/CD automation tool built into GitHub that lets you run workflows when events happen (like push, pull request).

Used for:

- Running tests
- Building code
- Deploying apps

GitHub Secrets are secure environment variables used to store sensitive data like:

- API keys
- Tokens
- Passwords

They are encrypted and not exposed in logs

Assignment: **End-to-End Multi-Modal Graph Based RAG System**

Course: Practical Design & Development of AI Applications

Marks: 20

Team Size: Max 4 students

Objective

Design and build a **production-style Multi-Modal Retrieval Augmented Generation (RAG) system with knowledge graph capabilities** that can process and retrieve information from **multiple data modalities (text, images, audio, video, etc.)** and generate meaningful responses.

Deadline

May 2nd.

Google Form Will be shared.

Assignment: Requirements

Full-Stack Application

- Frontend (React / Vue / etc.)
- Backend (FastAPI / Node.js / Flask)

Dockerization

- Both frontend and backend must run via Docker
- Use docker-compose.yml for orchestration

Multi-Modal Graph RAG

- Must support at least 3 modalities (e.g., text + images)
- Pipeline should include:
 - Query processing
 - Embedding generation
 - Vector database retrieval
 - Response generation using LLM

Version Control

- Code must be hosted on GitHub
- Use:
 - Meaningful commits
 - Branching strategy (feature branches, PRs)

Assignment: **Architecture Requirements**

Students should implement:

Ingestion Pipeline

- Text extraction (PDFs, docs)
- Image/audio preprocessing

Embedding Layer

- Text embeddings
- Image embeddings (CLIP / similar)
- Audio embeddings

Vector Database

- Pinecone / Weaviate

Retriever + Reranker

- Top-K retrieval
- Optional reranking

LLM Layer

- Context-aware response generation

Frontend

- Multi-modal input interface
- Response visualization

Assignment: Requirements

Presentation (10 minutes)

Must include:

- System architecture
- Live demo
- Challenges faced
- **Literature survey** (1 research paper) on:
 - AI Agents / Agentic Workflows / MCP

Assignment: Project Ideas

1. Multi-Modal Medical Assistant

- Input: Medical reports (PDF), X-ray images, symptoms (text)
- Output: Contextual insights + explanations

Possible Data Sources:

- NIH datasets
- Kaggle medical imaging datasets
- Open medical PDFs (research papers)

2. Smart Education Tutor

- Input: Lecture slides (PDF), recorded lectures (audio/video), student queries
- Output: Summaries, explanations, Q&A

Possible Data Sources:

- YouTube lectures
- Coursera course materials
- University open courseware

Assignment: Project Ideas

3. E-Commerce Product Intelligence System

- Input: Product images, descriptions, reviews
- Output: Smart search, recommendations, product Q&A

Possible Data Sources:

- Amazon product datasets (public dumps)
- Kaggle e-commerce datasets
- Web scraping (ethical & limited)

4. Legal/Policy Document Assistant

- Input: Legal documents (PDF), case summaries, scanned docs
- Output: Clause extraction, summaries, Q&A

Possible Data Sources:

- Indian Kanoon
- Government open data portals
- Public legal PDFs

Assignment: Project Ideas

5. Multi-Modal Log Analysis & Incident Assistant

Build an intelligent system that analyzes system logs, metrics, and visual dashboards to help detect issues, explain failures, and assist in debugging.

- Input: Text Logs, Metrics, Prometheus/Grafana Dashboard Screenshots
- Output: Root cause analysis (RCA), Error summarization

Possible Data Sources:

- Elasticsearch sample logs
- Kibana exported dashboards
- Prometheus metrics datasets
- Kaggle log anomaly datasets
- Synthetic logs generated using scripts (recommended for flexibility)

Assignment: Evaluation Rubrics

Evaluation Rubric (20 Marks)

Criteria	Marks	Description
System Design & Architecture	4	Clear, scalable RAG pipeline
Multi-Modal Implementation	5	Effective use of ≥ 2 modalities
Functionality & Demo	4	Working system with meaningful outputs
Dockerization & Deployment	2	Clean containerized setup
Code Quality & GitHub Usage	2	Structure, commits, documentation
Literature Survey	2	Understanding of research paper
Presentation Quality	1	Clarity, timing, explanation